1. Anagram Check
   INPUT str1, str2 IF length(str1) != length(str2) RETURN FALSE CREATE map charCounts FOR each char in str1: charCounts[char]++ FOR each char in str2: charCounts[char]-- FOR each count in charCounts: IF count != 0 RETURN FALSE RETURN TRUE

---

2. Decimal to Binary Conversion
   INPUT n IF n == 0 RETURN "0" binary = "" WHILE n > 0: binary = (n MOD 2) + binary n = n DIV 2 RETURN binary

---

3. Leap Year Check
   INPUT year IF (year MOD 4 == 0 AND year MOD 100 != 0) OR (year MOD 400 == 0) RETURN TRUE ELSE RETURN FALSE

---

4. Intersection of Sorted Arrays
   INPUT arr1, arr2 i = 0, j = 0, result = [] WHILE i < length(arr1) AND j < length(arr2): IF arr1[i] == arr2[j]: result.append(arr1[i]) i++, j++ ELSE IF arr1[i] < arr2[j]: i++ ELSE: j++ RETURN result

---

5. Remove Punctuation
   INPUT sentence cleaned = "" FOR each char in sentence: IF char is NOT punctuation: cleaned += char RETURN cleaned

---

6. String Length
   INPUT s count = 0 FOR each char in s: count++ RETURN count

---

7. Split Sentence into Words
   INPUT sentence words = [], current = "" FOR each char in sentence: IF char is space: IF current != "": words.append(current), current = "" ELSE: current += char IF current != "": words.append(current) RETURN words

---

8. Sort Words Alphabetically
   INPUT wordList FOR i from 0 to length-1: FOR j from i+1 to length-1: IF wordList[j] < wordList[i]: swap(wordList[i], wordList[j])

   RETURN wordlist

---

9. Find Target in Unsorted List
   INPUT list, target FOR i from 0 to length(list)-1: IF list[i] == target: RETURN i RETURN "Not Found"

---

10. Vowel Counter
    INPUT text counts = {a:0, e:0, i:0, o:0, u:0} FOR each char in lowercase(text): IF char in counts: counts[char]++ RETURN counts

---

11. Unique IP Addresses
    INPUT ipList uniqueSet = {} FOR ip in ipList: uniqueSet.add(ip) RETURN list(uniqueSet)

---

12. Main Diagonal Sum
    INPUT matrix (N x N) sum = 0 FOR i from 0 to N-1: sum += matrix[i][i] RETURN sum

---

13. Reverse String (Manual)
    INPUT s chars = array(s) left = 0, right = length(s)-1 WHILE left < right: swap(chars[left], chars[right]) left++, right-- RETURN join(chars)

---

14. Sum of Even Numbers
    INPUT list total = 0 FOR num in list: IF num MOD 2 == 0: total += num RETURN total

---

15. Second Largest in Array
    INPUT arr first = -∞, second = -∞ FOR num in arr: IF num > first: second = first, first = num ELSE IF num > second AND num != first: second = num RETURN second

---

16. GCD (Euclidean Algorithm)
    INPUT a, b WHILE b != 0: temp = b b = a MOD b a = temp RETURN a

---

17. Sum and Average
    INPUT list sum = 0 FOR num in list: sum += num avg = sum / length(list) RETURN sum, avg

---

18. Character Frequency
    INPUT s freq = {} FOR char in s: freq[char] = freq.get(char, 0) + 1 RETURN freq

---

19. Swap Without Temporary Variable
    INPUT a, b a = a + b b = a - b a = a - b RETURN a, b

---

20. Find All Divisors
    INPUT n divs = [] FOR i from 1 to n: IF n MOD i == 0: divs.append(i) RETURN divs

---

21. Even or Odd Check
    INPUT num IF num MOD 2 == 0: RETURN "Even" ELSE: RETURN "Odd"

---

22. Factorial
    INPUT n result = 1 FOR i from 2 to n: result *= i RETURN result

---

23. Score to Grade
    INPUT score IF score >= 80: RETURN "Distinction" ELSE IF score >= 60: RETURN "Merit" ELSE IF score >= 40: RETURN "Pass" ELSE: RETURN "Fail"

---

24. Password Validator
    INPUT pwd IF length(pwd) < 8: RETURN FALSE hasNum = FALSE, hasUpper = FALSE FOR char in pwd: IF char is digit: hasNum = TRUE IF char is uppercase: hasUpper = TRUE RETURN hasNum AND hasUpper

---

25. Longest Substring Without Repeating Chars
    INPUT s maxLen = 0, start = 0 seen = {} FOR i from 0 to length(s)-1: IF s[i] in seen AND seen[s[i]] >= start: start = seen[s[i]] + 1 seen[s[i]] = i maxLen = MAX(maxLen, i - start + 1) RETURN maxLen

---

26. Binary Tree Level-Order Traversal
    INPUT root IF root == null: RETURN [] queue = [root], result = [] WHILE queue not empty: levelSize = length(queue), currentLevel = [] FOR _ from 0 to levelSize-1: node = queue.pop(0) currentLevel.append(node.value) IF node.left: queue.append(node.left) IF node.right: queue.append(node.right) result.append(currentLevel) RETURN result

---

27. Rotate Matrix 90° Clockwise
    INPUT matrix (N x N) FOR i from 0 to N/2 - 1: FOR j from i to N-1-i-1: temp = matrix[i][j] matrix[i][j] = matrix[N-1-j][i] matrix[N-1-j][i] = matrix[N-1-i][N-1-j] matrix[N-1-i][N-1-j] = matrix[j][N-1-i] matrix[j][N-1-i] = temp

    RETURN matrix

28. Sliding Window Maximum
    INPUT arr, k result = [], deque = [] FOR i from 0 to length(arr)-1: IF deque[0] == i - k: deque.popFront() WHILE deque not empty AND arr[deque.back()] <= arr[i]: deque.popBack() deque.pushBack(i) IF i >= k - 1: result.append(arr[deque.front()]) RETURN result

---

29. Text Classification Model
    INPUT trainingTexts, labels tokens = tokenizeAndClean(trainingTexts) vectors = convertToTFIDF(tokens) model = trainClassifier(vectors, labels) RETURN model

---

30. Topic Modelling System
    INPUT corpus texts = preprocess(corpus) dtm = buildDocumentTermMatrix(texts) topics = applyLDA(dtm, numTopics) RETURN extractTopWords(topics)

---

31. Co-Reference Resolution
    INPUT text mentions = extractNounPhrasesAndPronouns(text) FOR each pronoun in mentions: candidates = filterCompatibleAntecedents(pronoun, mentions) antecedent = scoreAndSelect(candidates) link(pronoun, antecedent) RETURN resolvedMentions

---

32. Question Answering System
    INPUT context, query relevantPassages = retrievePassages(context, query) answer = extractAnswerSpan(relevantPassages, query) RETURN answer

---

33. Integer Square Root (Binary Search)
    INPUT n IF n < 2: RETURN n left = 1, right = n DIV 2, ans = 1 WHILE left <= right: mid = (left + right) DIV 2 IF mid * mid <= n: ans = mid, left = mid + 1 ELSE: right = mid - 1 RETURN ans

---

34. Intersection of Unsorted Lists
    INPUT list1, list2 set1 = set(list1) result = [] FOR item in list2: IF item in set1 AND item not in result: result.append(item) RETURN unique(result)

---

35. LRU Memory Buffer
    INIT capacity, cache = orderedDict() FUNCTION get(key): IF key in cache: moveToFront(cache, key), RETURN cache[key] ELSE RETURN null FUNCTION put(key, value): IF key in cache: moveToFront(cache, key), cache[key] = value ELSE: IF length(cache) == capacity: removeBack(cache) addToFront(cache, key, value)

---

36. Search Rotated Sorted Array
    INPUT arr, target left = 0, right = length(arr)-1 WHILE left <= right: mid = (left + right) DIV 2 IF arr[mid] == target: RETURN mid IF arr[left] <= arr[mid]: IF arr[left] <= target < arr[mid]: right = mid - 1 ELSE: left = mid + 1 ELSE: IF arr[mid] < target <= arr[right]: left = mid + 1 ELSE: right = mid - 1

    RETURN -1

37. Stack Using Array
    INIT stack = [], size = 0 FUNCTION push(x): stack.append(x), size++ FUNCTION pop(): IF size > 0: size--, RETURN stack.pop() ELSE ERROR FUNCTION peek(): IF size > 0: RETURN stack[size-1] ELSE ERROR

---

38. First Non-Repeating Character
    INPUT s freq = {} FOR char in s: freq[char] = freq.get(char, 0) + 1 FOR char in s: IF freq[char] == 1: RETURN char RETURN None

39. Detect Linked List Cycle
    INPUT head slow = head, fast = head WHILE fast != null AND fast.next != null: slow = slow.next fast = fast.next.next IF slow == fast: RETURN TRUE RETURN FALSE

---

40. Queue Using Two Stacks
    INIT s1 = [], s2 = [] FUNCTION enqueue(x): s1.push(x) FUNCTION dequeue(): IF s2 is empty: WHILE s1 not empty: s2.push(s1.pop()) RETURN s2.pop()

---

41. N Queens Problem
    INPUT N FUNCTION placeQueen(row, board): IF row == N: RETURN TRUE FOR col from 0 to N-1: IF isSafe(board, row, col): board[row][col] = 'Q' IF placeQueen(row + 1, board): RETURN TRUE board[row][col] = '.' RETURN FALSE RETURN placeQueen(0, emptyBoard(N))

---

42. Find Pairs Summing to Target
    INPUT arr, target seen = {}, pairs = [] FOR num in arr: comp = target - num IF comp in seen: pairs.add((MIN(num,comp), MAX(num,comp))) seen[num] = true RETURN unique(pairs)

---

43. Tower of Hanoi
    FUNCTION hanoi(n, source, target, aux): IF n == 1: PRINT "Move disk 1 from " + source + " to " + target ELSE: hanoi(n-1, source, aux, target) PRINT "Move disk " + n + " from " + source + " to " + target hanoi(n-1, aux, target, source)

---

44. Kadane's Algorithm (Max Subarray Sum)
    INPUT arr maxSoFar = -∞, currentMax = 0 FOR num in arr: currentMax += num IF maxSoFar < currentMax: maxSoFar = currentMax IF currentMax < 0: currentMax = 0 RETURN maxSoFar

---

45. Matrix Addition
    INPUT A, B (same dimensions R x C) INIT C as R x C matrix FOR i from 0 to R-1: FOR j from 0 to C-1: C[i][j] = A[i][j] + B[i][j]

    RETURN C

---

46. Matrix Multiplication
    INPUT A (M x N), B (N x P) INIT C as M x P matrix FOR i from 0 to M-1: FOR j from 0 to P-1: sum = 0 FOR k from 0 to N-1: sum += A[i][k] * B[k][j] C[i][j] = sum

    RETURN C

---

47. Inventory Update
    INPUT inventoryMap, transactions FOR t in transactions: IF t.type == "purchase": inventoryMap[t.item] += t.qty IF t.type == "delivery": inventoryMap[t.item] -= t.qty IF inventoryMap[t.item] < 0: RAISE "Insufficient Stock" RETURN inventoryMap

---

48. Recursive Folder Search
    FUNCTION search(directory, targetExt): results = [] FOR item in directory.contents: IF item is file AND item.ext == targetExt: results.add(item) ELSE IF item is directory: results.extend(search(item, targetExt)) RETURN results

---

49. Maze Pathfinding (BFS)
    INPUT maze, start, end queue = [(start, [start])], visited = {start} WHILE queue not empty: curr, path = queue.popFront() IF curr == end: RETURN path FOR neighbor in getValidNeighbors(curr, maze): IF neighbor not in visited: visited.add(neighbor) queue.append((neighbor, path + [neighbor]))

    RETURN "No Path"

50. Drone Coordination & Collision Avoidance
    INPUT drones, targets, safetyRadius WHILE not allDronesReachedTargets(): plannedPaths = [] FOR each drone in drones: path = computeAStar(drone.pos, drone.target) plannedPaths.append(path) adjustedPaths = resolveConflicts(plannedPaths, safetyRadius) FOR i from 0 to length(drones)-1: drones[i].moveAlong(adjustedPaths[i]) syncPositions() RETURN success